ParcelPilot — AI-Powered Logistics Support Agent
Architecture Note

Prince Kumar Sharma

Overview :

ParcelPilot is an AI-powered logistics support agent designed to help customers with order-related queries and operational actions. The system combines an LLM-based agent with deterministic tools, structured business data, document retrieval, access control, confirmation workflows, and escalation handling.

Agent Design:

The application uses an agent-based architecture where the user communicates through the application interface and requests are processed by the FastAPI backend.

The agent interprets the user's intent and determines whether the request requires:

- structured-data lookup,
- document/policy retrieval,
- an operational action,
- or escalation.

The agent can select the appropriate tool and use the returned information to formulate the final response.

The agent implementation is organized under:

```
app/agent/
├── agent.py
├── tools.py
└── tool_schemas.py
```

This separation keeps agent reasoning, tool definitions, and tool schemas organized independently.

Tool Design:

ParcelPilot separates tools according to their responsibility:

app/tools/

├── action_tools.py

├── data_tools.py

├── document_tools.py

└── tool_dispatcher.py

Data tools

Used for retrieving structured business information such as order and shipment information.

Document tools

Used to retrieve relevant information from the available operational and support documents.

Action tools

Used for operations such as cancellation and escalation preparation.

Sensitive actions are not blindly executed. The system can return a confirmation requirement before proceeding with an action. This provides an additional safety layer between the agent's reasoning and operational changes.

Structured Data and Documents :

Structured information is handled through the data layer:

app/data/

├── access_control.py

├── loader.py

└── service.py

The project includes structured assessment data and escalation information.

Document-based knowledge is handled through the RAG layer:

app/rag/

├── ingest.py

├── metadata.py

└── retriever.py

The document collection contains current and deprecated support policies, cancellation/SOP information, product operations information, and customer-specific agreements.

The system therefore separates structured operational data from unstructured policy and document knowledge

Source Reliability and Conflict Handling :

Document metadata is used to distinguish sources and support source-aware retrieval. Where multiple documents cover similar information, source characteristics such as document status and relevance are considered rather than treating every document as equally authoritative.

For example, the document set contains both current and deprecated policy versions. This allows the system to prioritize current operational guidance instead of blindly relying on an older document.

Structured business data is treated as the appropriate source for operational facts such as order information, while policy and procedural questions are handled through the document retrieval layer.

Access Control and Safety:

ParcelPilot includes an access-control layer and dedicated security tests.

app/data/access_control.py

The system also includes confirmation and security-related tests, including:

test_access_control.py

test_agent_security.py

test_confirmation.py

This helps prevent inappropriate access and ensures that sensitive actions are handled more cautiously than simple information retrieval.

Escalation:

When an issue cannot be appropriately resolved through normal customer-support workflows, ParcelPilot can prepare an escalation containing the relevant ticket/order information and reason.

The escalation functionality is separated from ordinary information retrieval, allowing the agent to distinguish between answering a question and initiating an operational support process.

API Layer:

FastAPI provides the application API:

app/api/

├── routes.py

└── schemas.py

This provides a structured interface between the user-facing application and the underlying agent and tool layers.

Testing:

The project includes dedicated tests covering the major components:

tests/

├── test_agent.py

├── test_agent_chat.py

├── test_agent_security.py

├── test_access_control.py

├── test_actions.py

├── test_confirmation.py

├── test_escalation.py

├── test_document_tools.py

├── test_retriever.py

├── test_api.py

└── test_tool_edge_cases.py

Testing covers agent behavior, tool execution, API behavior, security, confirmation workflows, escalation, retrieval, and edge cases.

Major Technical Trade-offs:

The main trade-off was between flexible LLM-based reasoning and deterministic operational behavior.

Using an LLM provides flexibility in understanding natural-language customer requests, but allowing the model to directly perform every operation would introduce unnecessary risk. ParcelPilot therefore uses the LLM primarily for intent understanding and tool selection while keeping operational logic inside dedicated tools.

Similarly, RAG provides flexible access to policy and operational documents, while structured data services remain responsible for transactional business information.

This hybrid approach provides a balance between natural-language flexibility, reliability, maintainability, and operational safety.